

American International University-Bangladesh (AIUB)

Department of Computer Science

Faculty of Science & Technology (FST)

Computer Graphics [A]

Final-Term Project Report

PROJECT TITLE:

Dreamland Village Ultimate

An Interactive 2D Animated Village Simulation Using OpenGL & C++

Semester:

Summer 2025-26

Supervised by:

Zishan Ahmed Onik

Group: 07		*Section: A*	
SL	**Student Name**	**Student ID**	***Contribution***
1	IFTEKHAR RAHMAN	23-55369-3	**Part 1:** Basic Setup, Primitives (`rect`, `circle`, `ellipse`, `line`, `drawText`), Sound Functions, Splash/Intro
2	NAJIAT ISLAM RISHAD	23-55218-3	**Part 2:** Village Landscape, Sky, Sun/Moon/Stars, Sun Rays, Twinkling Stars, Clouds, Birds, Hills, Paddy Field, Vegetation, Cottages, Smoke, Chimney
3	FAYSAL TALUKDER	23-55257-3	**Part 3:** Animals & Details, Cow, Wooden Fence, Chicken, Flowers, Flower Garden, Clothesline
4	MD. SAZZADUR RAHMAN	23-55247-3	**Part 4:** Hydrology & Weather, River, Ripples, Reflections, Bridge, Boat Body, Fisherman, Rowing Boat, Ducks, Fish, Fireflies, Rain Engine, Splashes
5	MD. ARAFAT RAHMAN SAMIR	23-55488-3	**Part 5:** Main Control & Pipeline, `display`, `update` loop, `toggleFullScreen`, `keyboard`, `specialKeys`, `mouse`, `init`, `main`

Contents

1 Introduction **2**
- 1.1 Project Overview . . . 2
- 1.2 Objectives . . . 2
- 1.3 Tools & Technologies Used . . . 2

2 Project Proposal **2**
- 2.1 Problem Statement . . . 2
- 2.2 Proposed Solution . . . 3

3 Schematic Diagram & Coordinate System **3**
- 3.1 Coordinate System Setup . . . 3
- 3.2 Layered Rendering Hierarchy . . . 3

4 List of Objects **4**

5 Functions to Represent the Objects **4**
- 5.1 Part 1: Low-Level Primitives & Base Engine (Iftekhar Rahman) . . . 4
- 5.2 Part 2: Village Landscape & Environment (Najiat Islam Rishad) . . . 5
- 5.3 Part 3: Animals & Village Details (Faysal Talukder) . . . 5
- 5.4 Part 4: River, Boat & Weather (Md. Sazzadur Rahman) . . . 5
- 5.5 Part 5: Main Control & Animation (Md. Arafat Rahman Samir) . . . 5

6 Algorithms & Mathematical Techniques Used **5**
- 6.1 Parametric Generation of Curves and Ellipses . . . 5
- 6.2 Harmonic Oscillations via Sine Functions . . . 6
- 6.3 Matrix Transformations & Pivot Rotations . . . 6

7 Interactive Functions & System Control **6**
- 7.1 Input Command Reference . . . 6
- 7.2 Frame Pipeline & Timer Dispatcher . . . 6

8 Scenario Descriptions & Output Screenshots **7**

9 Limitations & Future Improvements **9**
- 9.1 Current Limitations . . . 9
- 9.2 Future Enhancements . . . 9

10 Conclusion **9**

11 Source Code & Repository **9**

1 Introduction

1.1 Project Overview

Dreamland Village Ultimate is a real-time, interactive 2D graphical simulation developed using C++, modern OpenGL primitives, and the FreeGLUT toolkit. The project models an authentic representation of rural Bangladesh, synthesizing atmospheric cycles, indigenous flora, traditional architecture, livestock kinematics, and hydrological activities. Instead of relying on pre-rendered raster images or external textures, every visual entity is procedurally generated from low-level geometric primitives, including points, lines, polygons, and triangle fans within a unified orthographic coordinate space.

The simulation incorporates four distinct diurnal and meteorological modes: **Day**, **Sunset**, **Night**, and **Monsoon Rain**. Users can dynamically switch between these environments in real time. Each mode seamlessly alters the ambient chromatic palette, celestial trajectories, particle behaviors, and synchronized background acoustic tracks, while village kinematics continue without interruption.

1.2 Objectives

- Demonstrate practical proficiency in scan conversion, parametric curve rendering, and coordinate transformations.
- Render a culturally representative Bangladeshi village featuring tin-roof cottages, banyan and palm trees, crop fields, and riverside living.
- Implement mathematical motion patterns to simulate natural entity behaviors, including animal walks, avian wing flapping, traveling water waves, chimney smoke diffusion, and boat rowing.
- Construct an optimized, velocity-based particle system simulating monsoon downpours, river landing splashes, and randomized lightning flashes.
- Maintain a rock-solid 60 frames-per-second performance using timer-driven event callbacks without frame stuttering or memory leaks.

1.3 Tools & Technologies Used

Component	Specification / Purpose
Programming Language	C++ (Standard ISO C++11)
Graphics API	OpenGL (Core primitives rasterization), GLU (Orthographic projection)
Window / Event Toolkit	FreeGLUT 3.0+ (Windowing, event dispatcher, timer callback, bitmap typography)
Audio Architecture	Windows Multimedia Subsystem (`winmm.lib` / `PlaySoundA` for WAV playback)
Mathematical Engine	C++ Standard Math Library (`<cmath>`) for trigonometric and wave calculations
Compiler & IDE	MinGW GCC Compiler via Code::Blocks IDE (Windows 64-bit platform)
Version Control	Git and GitHub

2 Project Proposal

2.1 Problem Statement

Rural ecosystems comprise interconnected natural phenomena: non-linear light shifts, dynamic water surface reflections, layered foliage, and erratic living beings. Modeling these procedural aesthetics in pure 2D computer graphics without image assets presents a technical challenge requiring robust

mathematical formulations, back-to-front rendering management, and performance-conscious animation loops.

2.2 Proposed Solution

We propose a complete, multi-layered 2D procedural simulation driven by an event-driven loop. By dividing the project architecture into five distinct modular domains—primitives, environment, livestock, river/weather mechanics, and event controls—we deliver an interactive and optimized village simulation.

3 Schematic Diagram & Coordinate System

3.1 Coordinate System Setup

The application uses a 2D orthographic projection established via `gluOrtho2D(0, 1200, 0, 750)`, mapping a consistent world window of 1200×750 virtual units with the origin $(0, 0)$ anchored at the bottom-left corner.

3.2 Layered Rendering Hierarchy

To ensure correct visual depth without Z-buffer overhead, drawing functions are executed following a strict back-to-front painter's algorithm order:

1. **Celestial Canvas:** Sky gradient quad, sun/moon bodies, rotating rays, and drifting clouds ($Y \in [450, 750]$).
2. **Topographic Background:** Layered mountain silhouettes, green paddy fields, and detailed grass tufts ($Y \in [330, 500]$).
3. **Settlement & Flora:** Village trail, homestead fencing, cottages, chimney smoke, palm, banyan, and banana trees ($Y \in [240, 420]$).
4. **Living Entities & Micro-Props:** Grazing cow, pecking chicken, blooming flower garden, and clothesline ($Y \in [220, 310]$).
5. **Hydrologic Basin:** Flowing riverbed, dynamic trigonometric ripples, bridge, reflection shaders, swimming fish, and ducks ($Y \in [60, 240]$).
6. **Foreground & Interactive Craft:** Foreground riverbank, steerable wooden boat, and rowing fisherman ($Y \in [0, 150]$).
7. **Atmospheric Particle Overlay:** Dynamic rain streaks, river landing splashes, fireflies, and the on-screen HUD.

4 List of Objects

ID	Object Name	Base Primitives	Visual Description & Color Profile
OBJ-01	Sky Canvas	GL_QUADS	Multi-color gradient blending dynamically according to time-of-day.
OBJ-02	Sun & Sun-Rays	GL_TRIANGLE_FAN, LINES	Radiant golden disc with radially rotating projected line bursts.
OBJ-03	Moon & Stars	GL_POLYGON, GL_POINTS	Crescent moon with overlapping occluder circle; twinkling points.
OBJ-04	Drifting Clouds	Overlapping ellipse	Cluster of soft white/gray ellipses translating horizontally.
OBJ-05	Flying Birds	GL_TRIANGLES, LINES	Silhouette figures crossing the sky with oscillating wing flaps.
OBJ-06	Hills & Fields	GL_POLYGON	Distant mountain silhouettes and green-tinted crop fields.
OBJ-07	Village Cottages	GL_QUADS, POLYGON	Mud walls with corrugated tin roofs, windows, and active chimneys.
OBJ-08	Trees (Banyan/Palm)	Polygons & Ellipse Fans	Branching trunks supporting dense green layered leaf foliage.
OBJ-09	Grazing Cattle	Quads, Polygons, Lines	Detailed quadruped with cyclic walking legs and swinging tail.
OBJ-10	Flowing River	GL_QUADS, LINE_STRIP	Deep-blue aquatic basin animated with trigonometric ripple waves.
OBJ-11	Steerable Boat	GL_POLYGON, TRIANGLES	Traditional wooden dinghy with moving oar and seated fisherman.
OBJ-12	Rain Particles	GL_LINES, Points	Continuous falling line-particle streaks with ground impact ripples.

5 Functions to Represent the Objects

The codebase is partitioned into distinct functional modules matching the project structure:

5.1 Part 1: Low-Level Primitives & Base Engine (Iftekhar Rahman)

- `rect(x1, y1, x2, y2, r, g, b)`: Renders solid axis-aligned quadrilaterals for walls, paths, and structural members.
- `circle(cx, cy, r, segments)`: Approximates circular geometry via GL_TRIANGLE_FAN polar decomposition.
- `ellipse(cx, cy, rx, ry, segments)`: Parametric ellipse generator supporting differential aspect scaling.
- `line(x1, y1, x2, y2, width)`: Standardizes multi-width line rasterization across structural frames and fences.
- `drawText(x, y, text, font)`: Renders rasterized alphanumeric glyphs via `glutBitmapCharacter`.
- `Sound Functions`: Controls background ambient audio loops (`nature.wav`, `night.wav`, `rain.wav`, `cow.wav`).
- `drawIntro()`: Constructs the initial splash screen showcasing title cards, student IDs, and input keys.

5.2 Part 2: Village Landscape & Environment (Najiat Islam Rishad)

- `drawSky()`, `drawSunMoonStars()`, `drawSunRays()`, `drawTwinklingStars()`: Manages the celestial canvas, diurnal transitions, radiant solar bursts, and nocturnal stars.
- `drawCloud()`, `drawClouds()`, `drawBird()`, `drawBirds()`: Translates overlapping cloud clusters and flaps bird wings across the horizon.
- `drawHills()`, `drawPaddyField()`, `drawGrassDetails()`, `drawVillagePath()`: Renders the multi-tier mountain ridges, green crop planes, and organic dirt tracks.
- `drawPalmTree()`, `drawBanyanTree()`, `drawBananaPlant()`: Procedurally models trunk geometry and dense layered leaf canopies with wind sway.
- `drawHouseOne()`, `drawHouseTwo()`, `drawSmoke()`, `drawChimney()`: Generates traditional tin-roofed houses, active brick chimneys, and ascending smoke particles.

5.3 Part 3: Animals & Village Details (Faysal Talukder)

- `drawCow()`: Animated quadruped cow model featuring rotating limb armatures, swinging tail, and grazing kinematics.
- `drawChicken()`: Animated hen with cyclic pecking and walking behaviors around the courtyard.
- `drawFence()`, `drawFlower()`, `drawFlowerGarden()`, `drawClothesline()`: Micro-props enriching the homestead area with swaying flower heads and drying fabrics.

5.4 Part 4: River, Boat & Weather (Md. Sazzadur Rahman)

- `drawRiver()`, `drawRiverRipples()`, `drawReflection()`, `drawBridge()`: Blue river basin rendering dynamic surface sine ripples, mirrored shadows, and connecting bridge.
- `drawBoatBody()`, `drawFisherman()`, `drawBoat()`: Interactive rowing boat with articulated fisherman and synchronized oar strokes.
- `drawDuck()`, `drawDucks()`, `drawOneFish()`, `drawFish()`: Floating river waterfowl and jumping fish arc paths.
- `drawFireflies()`, `drawRain()`, `drawRainSplashes()`: Velocity-based particle systems simulating night fireflies, monsoon rain streaks, and ground impacts.

5.5 Part 5: Main Control & Animation (Md. Arafat Rahman Samir)

- `display()`: Primary rendering callback executing back-to-front scene draws.
- `update()`: Recursive 60 FPS timer callback updating motion phases and particles.
- `toggleFullScreen()`: Toggles standard window and hardware fullscreen displays.
- `keyboard()`, `specialKeys()`, `mouse()`: Comprehensive event listeners handling mode transitions and boat navigation.
- `init()`, `main()`: OpenGL context initialization, projection setup, and message pump launch.

6 Algorithms & Mathematical Techniques Used

6.1 Parametric Generation of Curves and Ellipses

Smooth circular and elliptical curves are rasterized using incremental angular steps $\theta_i = \frac{2\pi i}{N}$:

$$x_i = c_x + r_x \cdot \cos(\theta_i), \qquad y_i = c_y + r_y \cdot \sin(\theta_i) \quad \text{for } i \in [0, N] \tag{1}$$

Setting $r_x = r_y$ produces standard circles; distinct values yield varied elliptical foliage, ripples, and water lilies.

6.2 Harmonic Oscillations via Sine Functions

Periodic kinetic behaviors throughout the scene are governed by sinusoidal functions parameterized by accumulated phase variables updated every frame:

$$y(t) = y_0 + A \cdot \sin(\omega t + \phi) \tag{2}$$

- **Traveling River Waves:** $y_{\text{water}} = 180.0 + 3.5 \cdot \sin(0.025x + 3.2t)$ produces fluid surface propagation.
- **Cattle Locomotion:** Armature swings obey $\theta_{\text{leg}} = A \cdot \sin(\text{phase})$, driving reciprocal limb movement.
- **Foliage Wind Sway:** Leaf tip offsets vary harmonically as $\Delta x = \pm A \cdot |\sin(\text{phase})|$.

6.3 Matrix Transformations & Pivot Rotations

Matrix stacks isolate local object transformations. Sun ray bursts and oar pivots use local translation and rotation:

$$P' = T(x_{\text{pivot}}, y_{\text{pivot}}) \cdot R(\theta) \cdot T(-x_{\text{pivot}}, -y_{\text{pivot}}) \cdot P \tag{3}$$

7 Interactive Functions & System Control

Part 5 provides real-time event dispatchers for input handling, window reconfiguration, and frame timing.

7.1 Input Command Reference

Input Trigger	System Response / State Modification
`Enter`	Dismisses introduction splash screen and activates real-time simulation.
`1` / `D`	Activates **Day Mode**: Golden sunlight, daytime sky gradient, playing `nature.wav`.
`2`	Activates **Sunset Mode**: Warm crimson-orange horizon with descending evening sun.
`3` / `N`	Activates **Night Mode**: Dark starry sky, glowing moon, fireflies, playing `night.wav`.
`4` / `R`	Activates **Rain Mode**: Overcast gloom, falling rain particles, playing `rain.wav`.
`B / b`	Toggles autonomous boat rowing animation.
`L / l`	Toggles illuminated window light fixtures in village cottages.
`H / h`	Displays or dismisses on-screen HUD navigation help menu.
`F / f`	Toggles display between standard resolution and Fullscreen mode.
`M / U`	Mutes / un-mutes ambient background audio playback.
`Left / Right Arrow`	Manually steers boat along river navigation paths.
`Mouse Left / Right`	Accelerates boat forward or triggers reverse propulsion.
`Esc`	Closes application safely and deallocates FreeGLUT resources.

7.2 Frame Pipeline & Timer Dispatcher

Simulation states advance via a recursive GLUT timer callback running at approximately 60 FPS:

```
glutTimerFunc(16, update, 0);
```

The function increments positional coordinates, steps animation phases, handles particle recycling, and issues `glutPostRedisplay()` calls without dropping frames.

8 Scenario Descriptions & Output Screenshots

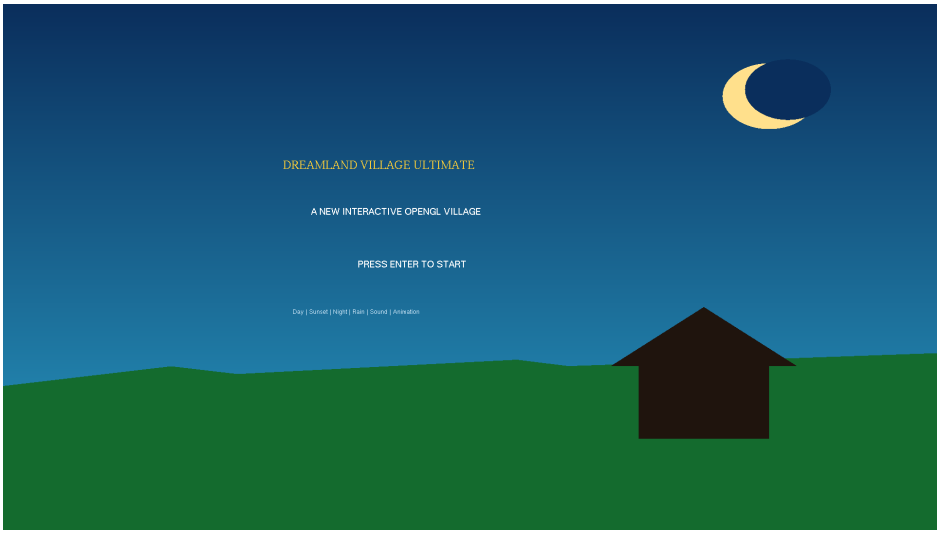


Figure 1: **Introductory Scene:** Title banner, team credits, and initial start instructions.



Figure 2: **Daytime Village Scene:** Clear daylight sky, radiant sun, green paddy fields, cottages, and walking livestock.



Figure 3: **Sunset Mode:** Warm orange-red horizon, evening sun dip, and soft landscape illumination.



Figure 4: **Night Simulation:** Deep dark sky, twinkling stars, glowing crescent moon, fireflies, and lit cottage windows.



Figure 5: **Monsoon Rain Scenario:** Dark overcast skies, angled rain streaks, water landing ripples, and storm audio.

9 Limitations & Future Improvements

9.1 Current Limitations

- **Kinematic Motion:** Craft and animal trajectories follow fixed procedural paths rather than true dynamic collision response systems.
- **Single-File Architecture:** The complete codebase is structured in a monolithic format rather than distinct polymorphic object-oriented classes.
- **Platform-Dependent Audio:** Audio playback relies on Windows Multimedia calls (`winmm.lib`), which requires alternative backend wrappers on POSIX/Linux distributions.

9.2 Future Enhancements

- Port audio playback to OpenAL or SDL2_mixer to ensure native cross-platform support.
- Migrate the 2D scene into 3D by adding perspective projection, camera orbits, and Phong illumination shaders.
- Implement AI pathfinding for village characters and wandering livestock.
- Introduce dynamic seasonal weather transitions (e.g., Autumn Kashful blooms, winter morning fog).

10 Conclusion

Dreamland Village Ultimate successfully demonstrates core 2D computer graphics concepts through modular OpenGL programming, delivering an optimized real-time simulation at 60 FPS.

11 Source Code & Repository

The complete source code and build dependencies are available at:

`https://github.com/munshi-rishad/ComputerGraphics-Dreamland-Village-Ultimate`